Daniel Miller

Assignment 3: Warehouse Crate Stacking Problem

Resources I used to complete this assignment:

[1] “Python Program for Tower of Hanoi,” *GeeksforGeeks*, May 23, 2014. https://www.geeksforgeeks.org/python-program-for-tower-of-hanoi/

[2] “21.1: The Towers of Hanoi,” *Engineering LibreTexts*, Jun. 29, 2021. https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_and_Computation_Fundamentals/Mathematics_for_Computer_Science_(Lehman_Leighton_and_Meyer)/05%3A_Recurrences/21%3A_Recurrences/21.01%3A_The_Towers_of_Hanoi

[3] “Tower of Hanoi Puzzle,” *Enjoymathematics.com*, 2020. https://www.enjoymathematics.com/blog/tower-of-hanoi-puzzle

a)

I recognized that this algorithm was the classic tower of hanoi problem, which is an algorithm I’ve worked with before.

This function is broken down recursivly by n-1 crates (whihc is solved for until the base case is reached) where in each recursive call it facitlitates the transfer of crates to different locations where only one crate can be moved and a heavier crate cannot be placed on a smaller crate, the crates of amount n need to be stacked in decreasing order of weight at first and that is assumed in this case

The base case is that one crate needs to be moved to the destination

At the source stack A you’d move the top n-1 crate to the auxiliary stack B using the source C as a temporary placeholder

Then you’d move the heaviest crate available frome stack A to stack C the destination

Then you’d move the n-1 crate from rack B to rack C using rack A as a temporary placeholder and then this should all repeat

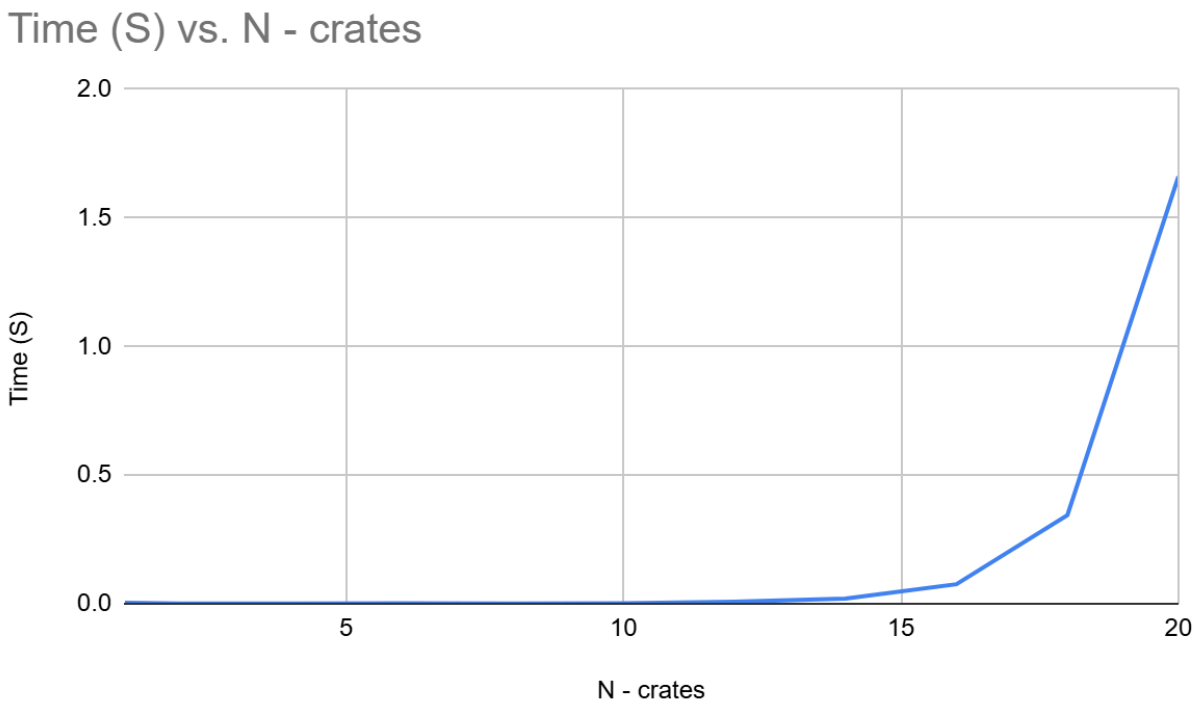
Pseudo code for this problem should look something like this:

```
moveCrates(n, source, auxiliary, destination):
        if n == 1:
                print("Move crate 1 from rack source to rack destination")
```

```
else:
        moveCrates(n-1, source, destination, auxiliary)
        print("Move crate n from rack source to rack destination")
        moveCrates(n-1, auxiliary, source, destination)
```

b)

Here is the chart that was created by analyszing the runtimes of the algorithm as n is increasing:



From observing the chart it seems like the algorithm has somewhere between a O(n^2) or O(2^n) pattern, the inputs seem to be to small for my computer to make an accurate statement from the chart about which big O trend it follows, however knowing the algorithm allows me to know that this algorithm should run a big O of O(2^n).

c)

The recurrence equation for the tower of hanoi algorithm should look like this where T(n) is the value of the number of moves in the problem with n disks

Proof by recurrence tree

T(n)

T(n-1) T(n-1)

T(n-2) T(n-2) T(n-2) T(n-2)

T(1) T(1) T(1) T(1) T(1) T(1) T(1) T(1)

Number of recursive calls grow exponentially each level i.e first level = 2secod = 4 third = 8 and so on

Total number of calls = 1 + 2 +4 + 8 …. N times = 2^n-1

So total number of operations = 2^n -1 = O(2^n)

Proof by induction

T(n) = 2^n -1

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2 * T(n-1) + 1, & \text{if } n > 1 \end{cases}$$ = T(n) = 2^n -1 or O(2^n) in big O

For T(n) >= 2

T(n) = 2T(n-1) + 1
= 2(2^(n-1) -1) + 1
= 2^n - 1

d)

I'm not the most familiar with Bitwise operations or Memoization however from what I've read I don't believe either would be able to optomize the algorithm too much as the mathematical proof for the algorithm is 2^(n) - 1 and there can't be fewer moves thus neither of these will change the O(2^n) time complexity. It seems for memoizastion it caches repeated sequences from recursion and uses the cache to speed the process up however this works when a recursive loop has some sort of overlapping subproblems and the hanoi algorithm is deterministic and has a set sequence of steps for the optomized scenario. For Bitwise operations it seems to me that there may be some use of it in choosing direction of where the crates should go however I don't think it would reduce the complexity and thus I don't believe that it would reduce the time of the algorithm by much if any.